Network Detection & Response (NDR) Platform

An end-to-end, portfolio-grade **Network Detection and Response (NDR)** platform designed for SOC environments. Ingests raw network traffic, performs dual-path classification combining **Suricata signature detection**, an **XGBoost supervised flow classifier**, and a **PyTorch benign-trained**

Autoencoder, and enforces automated firewall containment actions (**OPNsense REST API** / host `iptables`) under a strict **fail-secure guarantee**.

System Architecture

```
flowchart TD
    subgraph Ingestion ["1. Traffic Ingestion Layer"]
        A["Raw Traffic / Replayed PCAPs"] ⟶ B["Zeek Sensor"]
        A ⟶ C["Suricata Sensor"]
        B ⟶ D["ZeekParser (conn/dns/ssl/http)"]
        C ⟶ E["SuricataParser (eve.json)"]
    end

    subgraph FeatureEng ["2. Feature Engineering"]
        D ⟶ F["FlowFeatureExtractor (31+ Tabular Features)"]
        D ⟶ G["Shannon & DNS Subdomain Entropy"]
        D ⟶ H["Port Distribution Entropy"]
        D ⟶ I["Inter-Arrival Timing Stats"]
        F & G & H & I ⟶ J["NormalizedFlow Common Schema"]
    end

    subgraph DualPath ["3. Dual-Path Detection Engine"]
        E ⟶ K["Suricata Signature Engine (ET Rules)"]
        J ⟶ L["Supervised Flow Classifier (XGBoost)"]
        J ⟶ M["Benign Baseline Autoencoder (PyTorch MSE)"]
        K ⟶|"Signature Matches"| N["Detection Signals"]
        L ⟶|"Class Probabilities"| N
        M ⟶|"Reconstruction Loss"| N
    end

    subgraph Decision ["4. Fail-Secure Decision Engine"]
        N ⟶ O["Decision Arbiter"]
        P["Timeout & Crash Watchdog"] -.→|"fail_secure_guard"| O
        O ⟶ Q{"Verdict Selection"}
        Q ⟶|"Confidence ⩾ 0.85 / Critical Sig"| R["BLOCK_AND_ISOLATE"]
        Q ⟶|"Suspicious / Anomaly"| S["QUARANTINE_VLAN (VLAN 99)"]
        Q ⟶|"Low Risk / Benign"| T["PASS"]
        Q ⟶|"Any Component Failure / Timeout"| R
    end

    subgraph Response ["5. Containment & Visibility"]
        R & S ⟶ U["ContainmentManager"]
        U ⟶|"Primary Action"| V["OPNsense Firewall REST API"]
        U ⟶|"Failover Action"| W["Local Host iptables Drop"]
        O ⟶ X["SIEM Dispatcher (ECS JSON)"]
        X ⟶ Y["Wazuh / ELK Dashboard"]
    end
```

Core Design Principles

1. **Fail-Secure Architecture**: If the classifier or autoencoder throws, times out (>500ms), or an API is unreachable, the verdict defaults to defensive containment (`BLOCK_AND_ISOLATE` with `fail_secure=True`)—**never a silent bypass**.
2. **Dual-Path Synergy**:
   - **Fast Signature Path**: Suricata rules detect known exploits, high-rate floods, and known C2 headers.
   - **Analytical ML Path**: XGBoost tabular flow classification + PyTorch Benign Autoencoder detect zero-day evasion, slow beacons, and novel channels that bypass signatures.
3. **Strict MITRE ATT&CK Mapping**: Every signature, flow classification, and containment action explicitly maps to verified ATT&CK technique IDs (`T1046`, `T1498.001`, `T1071.001`, `T1071.004`, `T1572`, `T1110.001`).
4. **Empirical Honesty (Zero Fabrication Guarantee)**: All metrics reflect real holdout test evaluations against 100,000+ real network flows from CICIDS2017.

Empirical Results & Benchmarks

Full evaluation details are documented in `docs/results.md`.

Supervised Flow Classifier (Evaluated on 20,002 Unseen Test Flows)

Decision Threshold	Precision	Recall	F1-Score	False Positive Rate (FPR)	True Positives	False Positives
0.50	0.9998	0.9996	0.9997	0.0004	12,234	3
0.70	0.9998	0.9996	0.9997	0.0004	12,234	3
0.85 (Production Setting)	**0.9998**	**0.9995**	**0.9996**	**0.0004**	**12,233**	**3**
0.95	0.9999	0.9990	0.9995	0.0001	12,227	1

Dual-Path Synergy: What ML Catches When Signatures Miss

Attack Scenario	Signature Alert?	ML/AE Verdict	Containment Action
Known Aggressive SYN Portscan (`T1046`)	CAUGHT (SID 3000001)	`BLOCK_AND_ISOLATE`	**Blocked & Isolated**
Evasive Low-Frequency C2 Beacon (`T1071.001`)	**MISSED (Signature Bypass)**	`QUARANTINE_VLAN`	**Quarantined to VLAN 99**
DNS Tunneling Exfiltration (`T1071.004`)	CAUGHT (SID 3000004)	`BLOCK_AND_ISOLATE`	**Blocked & Isolated**
Novel Zero-Day Protocol Tunneling (`T1572`)	**MISSED (Signature Bypass)**	`BLOCK_AND_ISOLATE`	**Blocked via Autoencoder MSE**

Installation & Setup

Prerequisites

- Python 3.11+ (Python 3.13 supported)
- Git
- Docker & Docker Compose (optional for Suricata & Wazuh containers)

1. Clone & Set Up Environment

```
git clone https://github.com/your-username/ndr-platform.git
cd ndr-platform

# Create and activate virtual environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1  # Windows PowerShell
# source .venv/bin/activate    # Linux / macOS

# Install all dependencies
pip install -e .
```

2. Run Test Suite

```
pytest
```

Expected: 20 passed tests covering loaders, entropy, classification, fail-secure guards, and firewall failover.

Execution Guide

1. Ingest Raw Datasets & Extract Features

Place raw CSV/PCAP files in `data/raw/` and run:

```
python scripts/build_dataset.py
```

Generates `data/processed/processed_flows.csv` and `data/processed/manifest.json`.

2. Train Models & Run Holdout Evaluation

```
python scripts/train_models.py
python scripts/eval/evaluate_classifier.py
```

3. Run Dual-Path Detection & Signature Comparison

```
python scripts/eval/test_signatures.py
```

4. Generate Synthetic Attack PCAPs

```
python scripts/generate_test_traffic/generate_attacks.py
```

5. Simulate Atomic Red Team Attack Traffic

```
python scripts/generate_test_traffic/atomic_runner.py --technique T1071.001 --target 192.168.
```

Repository Structure

```
ndr-platform/
├── docs/                          # Architecture, threat mapping, live validation runbooks, res
├── lab/                           # VM provisioning scripts, network diagrams, and setup notes
├── data/                          # Raw datasets, processed feature CSVs, and PCAPs
├── src/ndr/
│   ├── ingest/                    # CICIDS/UNSW loaders, Zeek TSV/JSON parser, Suricata parser
│   ├── features/                  # 31+ tabular flow features, Shannon/DNS/port entropy, timing
│   ├── detection/
│   │   ├── signatures/            # Suricata engine & custom MITRE-mapped rules
│   │   ├── classifier/            # Supervised XGBoost flow classifier
│   │   └── anomaly/               # PyTorch Benign Autoencoder
│   ├── decision/                  # Fail-secure arbiter & watchdog guard
│   ├── response/                  # OPNsense REST API client, iptables fallback, SIEM dispatche
│   └── viz/                       # Metrics reporter & evaluation utilities
├── models/                        # Serialized trained models & feature importances
├── scripts/                       # Dataset builder, training, evaluation, attack generators
└── tests/                         # 20 unit and integration tests
```

License

MIT License. Open for academic research and portfolio review.